# Reproducing ParticleNet for Jet Tagging

Xinzhou Guo
PHY 506 Computational Physics II

May 2026

**Abstract**

Jet tagging is an important task in high-energy particle physics, aiming to identify the origin of jets produced in proton-proton collisions at the Large Hadron Collider (LHC). Traditional machine learning approaches often represent jets as images or sequences, which introduce artificial structures on fundamentally unordered particle systems. In this project, the ParticleNet architecture was studied and reproduced following the CMS Machine Learning documentation. ParticleNet represents jets as particle clouds and applies graph neural network techniques through EdgeConv and dynamic graph construction. Multiple models including a multilayer perceptron (MLP), DeepAK8, and ParticleNet were trained and evaluated using CMS jet-tagging datasets. Performance was analyzed using ROC curves, AUC scores, and TensorBoard monitoring of training and evaluation losses. The reproduced results demonstrate that ParticleNet achieves the best overall jet-tagging performance among the tested models. A brief outlook toward transformer-based architectures such as GloParT is also discussed.

## Contents

# 1 Introduction

Jets are collimated sprays of particles produced in high-energy collisions at the Large Hadron Collider (LHC). Because quarks and gluons cannot exist as free particles due to quantum chromodynamics (QCD), they hadronize into observable particle showers known as jets. Identifying the origin of jets, such as quark jets, gluon jets, or boosted heavy-particle jets, is a fundamental challenge in experimental high-energy physics.

Machine learning techniques have become increasingly important in jet tagging tasks. Traditional approaches often represent jets as images for convolutional neural networks (CNNs) or as ordered sequences for recurrent neural networks (RNNs). However, jets are fundamentally unordered particle systems, and these representations impose artificial structures on the data.

ParticleNet addresses this issue by representing jets as particle clouds, treating jets as unordered sets of particles and applying graph neural network methods to learn local and global particle correlations. Figure 1 illustrates the general jet-tagging workflow at the LHC, from collision events to jet reconstruction and final classification.

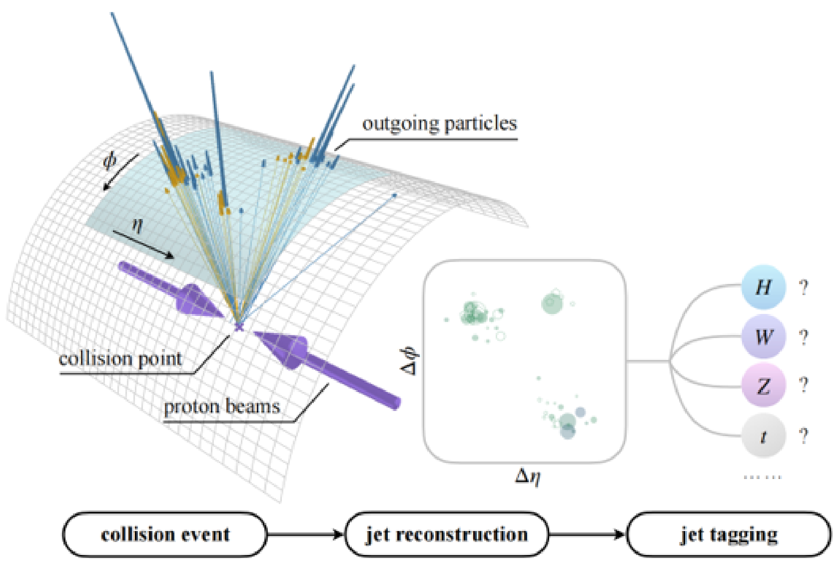


Figure 1: Illustration of jet tagging at the CERN Large Hadron Collider (LHC). Jets produced from proton-proton collisions are reconstructed and classified according to their origin particles. Figure adapted from Qu et al. [5].

# 2 ParticleNet Architecture

## 2.1 Particle Cloud Representation

ParticleNet represents each jet as a particle cloud, where each particle is described by a feature vector containing kinematic and geometric information. Unlike image-based or sequence-based approaches, the particle cloud representation preserves permutation invariance and avoids introducing artificial ordering into the data.

This representation is conceptually related to point-cloud learning methods in computer vision and to machine learning frameworks such as Deep Sets and Message Passing Neural Networks (MPNNs).

## 2.2 EdgeConv and Dynamic Graph Construction

ParticleNet is based on the Dynamic Graph CNN (DGCNN) framework and uses the EdgeConv operation to aggregate information from neighboring particles. As illustrated in Figure 2, point-cloud convolution differs fundamentally from ordinary image convolution because neighboring particles are not arranged on a fixed regular grid.

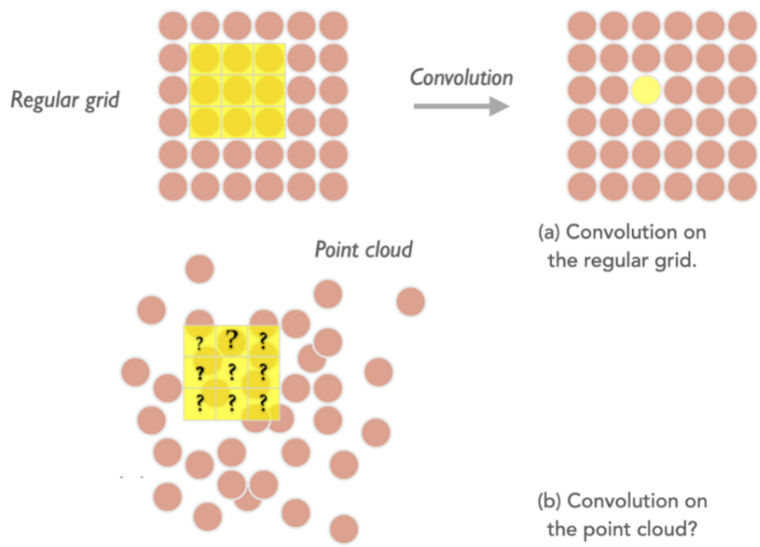


Figure 2: Comparison between convolution on regular grids and convolution on point clouds. Unlike image grids, particle clouds require graph-based neighborhood aggregation methods such as EdgeConv. Figure adapted from the ML4Jets ParticleNet material and CMS ParticleNet documentation [2].

For each particle $i$, the algorithm identifies its $k$-nearest neighbors and computes edge features describing the relationship between neighboring particles. These edge features are passed through neural network layers and aggregated to update particle representations. The EdgeConv update for a particle $i$ can be written schematically as

$$x_i' = \frac{1}{k} \sum_{j=1}^{k} h_\Theta(x_i, x_j - x_i), \tag{1}$$

where $x_i$ represents the feature vector of the central particle, $x_j$ are neighboring particle features, and $h_\Theta$ is a learnable neural-network transformation. The aggregation over neighboring particles preserves permutation invariance. Figure 3 shows how local particle neighborhoods are aggregated through the EdgeConv operation.

The graph structure is dynamically updated after each EdgeConv block. In the first layer, neighbors are defined using physical coordinates such as $(\Delta\eta, \Delta\phi)$. In later layers, the learned feature vectors themselves are treated as coordinates in a latent feature space, generating a latent graph that evolves during training. Therefore, the latent graph is not a fixed physical detector graph. Instead, it is a dynamically learned graph in feature space, where particle connections are continuously updated according to learned representations. The internal structure of the EdgeConv block is shown in Figure 4.

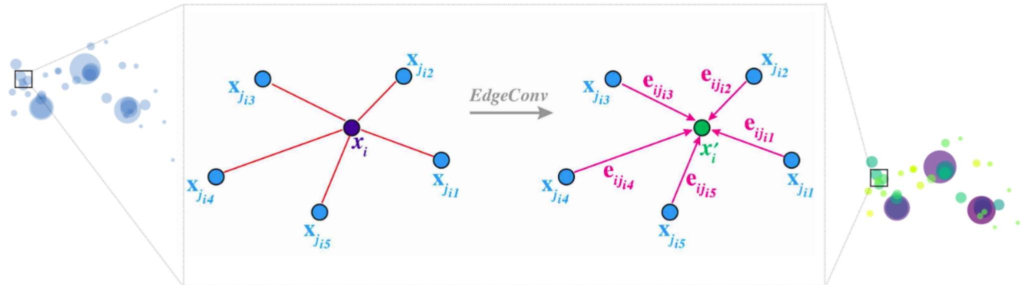


Figure 3: Illustration of the EdgeConv operation. For each central particle, local neighbor relations are constructed and transformed into learned edge features. Figure adapted from the CMS ParticleNet documentation [2].

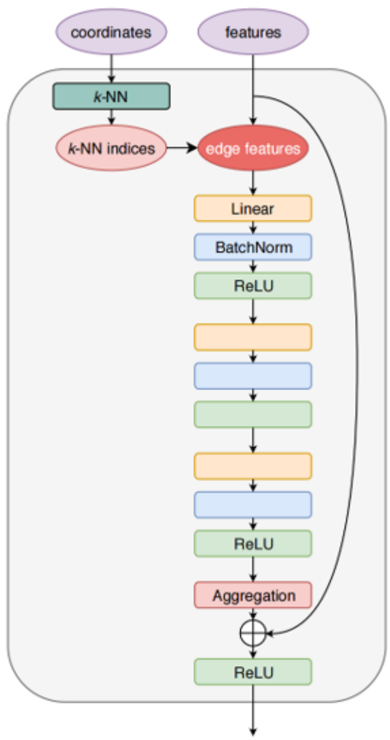


Figure 4: Structure of the EdgeConv block used in ParticleNet. Neighboring particles are aggregated through learned edge features and graph-based convolutions. Figure adapted from Qu and Gouskos [1].

## 2.3 ParticleNet Structure

The ParticleNet model consists of multiple stacked EdgeConv blocks followed by global average pooling and fully connected classification layers. The EdgeConv blocks progressively learn higher-level jet substructure features, while the pooling operation aggregates particle-level information into a jet-level representation. The full ParticleNet architecture is shown in Figure 5.

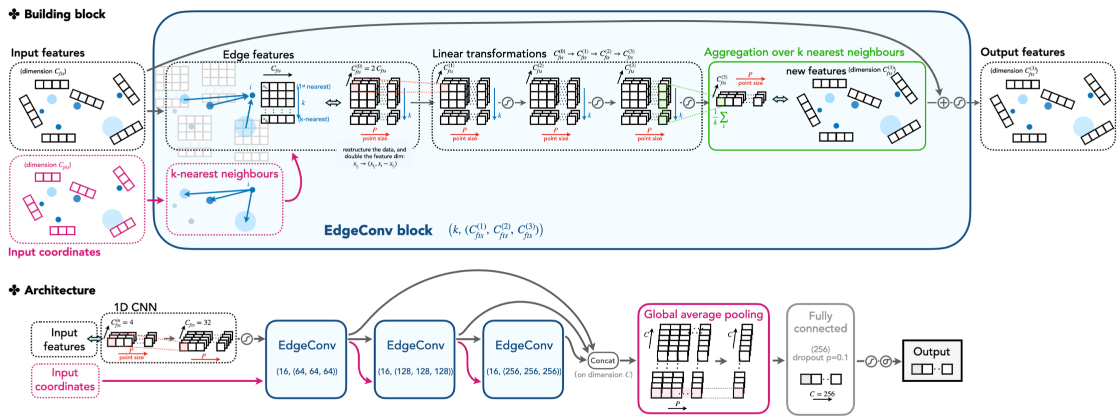


Figure 5: Full ParticleNet architecture. Multiple EdgeConv blocks progressively learn hierarchical jet-substructure features before global pooling and final classification layers. Figure adapted from the CMS ParticleNet documentation [2] and Qu and Gouskos [1].

# 3 Methodology

## 3.1 Dataset and ROOT Files

The models were trained and evaluated using the CMS jet-tagging datasets provided in the ParticleNet documentation and related CMS studies. ROOT files containing jet constituent information and labels were used for training, evaluation, and testing.

The dataset includes low-level particle information such as kinematic variables, tracks, and energy depositions, which are used as input features for machine learning models.

## 3.2 Models and Training

Three models were studied and compared:

- Multilayer Perceptron (MLP)
- DeepAK8 (1D CNN)
- ParticleNet (DGCNN)

Training and evaluation were performed following the CMS Machine Learning documentation. Hyperparameter tuning included optimizer selection, learning-rate adjustment, and model-size comparison. The primary goal of this project was not to develop a new jet-tagging architecture, but to reproduce and understand the ParticleNet workflow. The training procedures, evaluation pipelines, ROC-curve generation, and TensorBoard monitoring were implemented and reproduced following the CMS ParticleNet documentation.

The Ranger optimizer, which combines RAdam and LookAhead methods, was studied alongside AdamW optimization. TensorBoard was used throughout training to monitor optimization behavior, as shown in Figure 6.

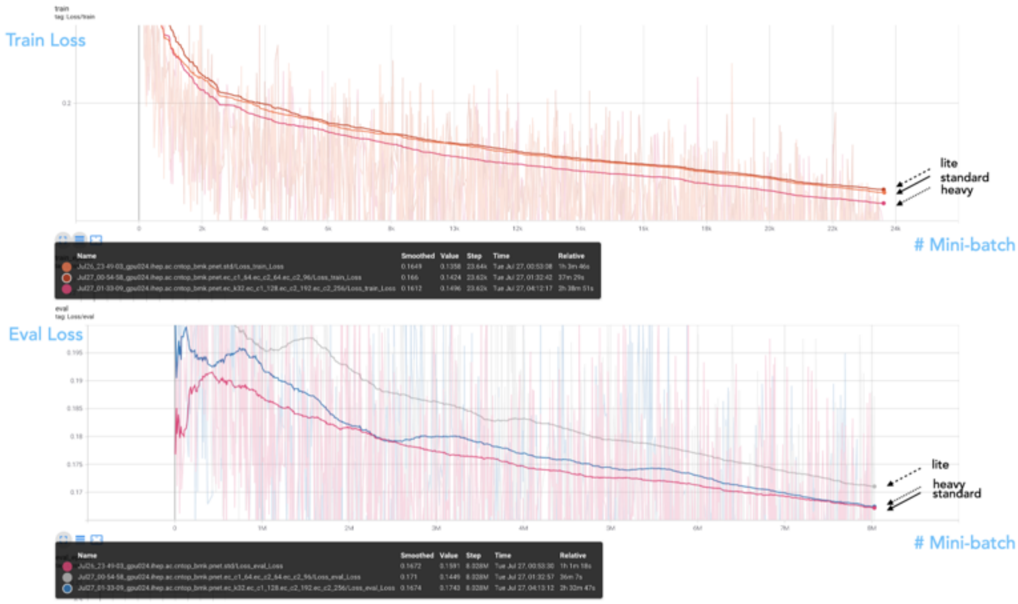


Figure 6: TensorBoard monitoring plots for training and evaluation losses. Larger model configurations achieve lower training losses but begin to show signs of mild overfitting as evaluation losses stop improving proportionally. Figure adapted from the CMS ParticleNet documentation [2].

### 3.3 Evaluation Metrics

Model performance was evaluated using Receiver Operating Characteristic (ROC) curves, Area Under the Curve (AUC), classification accuracy, and background rejection at fixed signal efficiency.

TensorBoard was additionally used to monitor training loss, evaluation loss, convergence behavior, and possible overfitting during training.

## 4 Results

### 4.1 Model Comparison

The reproduced ROC curves show that ParticleNet achieves the best overall jet-tagging performance among the tested models. Compared to the MLP and DeepAK8 models, ParticleNet demonstrates improved signal-background discrimination due to its graph-based representation learning. The reproduced ROC comparison is shown in Figure 7.

### 4.2 Hyperparameter Tuning

Learning-rate tuning and optimizer selection significantly affected training stability and convergence. The Ranger optimizer generally provided more stable training behavior and better convergence than standard AdamW optimization in this setup.

### 4.3 TensorBoard Monitoring

TensorBoard monitoring was used to study training and evaluation losses throughout optimization. Larger model configurations achieved lower training losses but showed signs of mild overfitting, where evaluation losses stopped improving proportionally. The corresponding training and evaluation losses are shown in Figure 8.

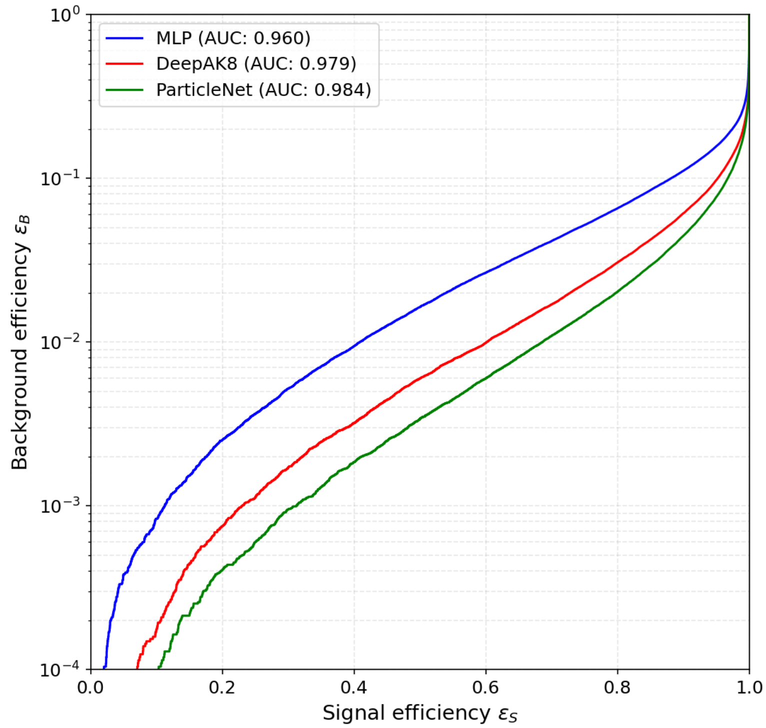


Figure 7: Reproduced ROC curve comparing the performance of MLP, DeepAK8, and ParticleNet models on the top-tagging benchmark dataset. The models were trained and evaluated using the CMS ParticleNet workflow [2,6]. ParticleNet achieves the strongest overall background rejection and classification performance.

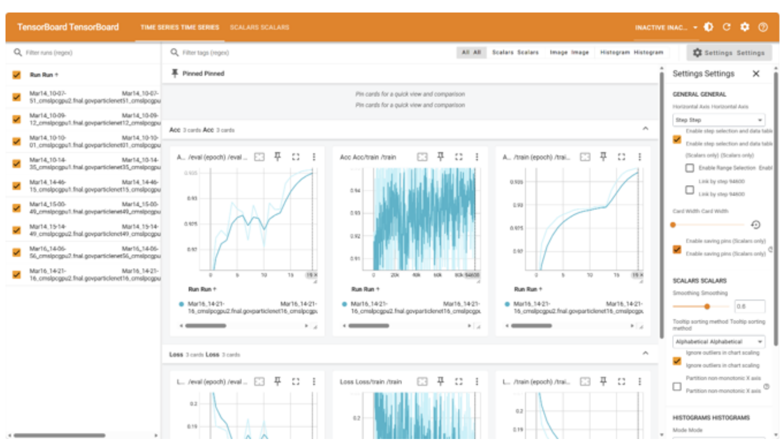

Figure 8: TensorBoard monitoring plots for training and evaluation losses. Larger model configurations achieve lower training losses but begin to show signs of overfitting as evaluation losses stop improving proportionally.

## 5 Future Outlook: GloParT

As a future direction, transformer-based architectures such as Particle Transformer and GloParT were explored. Unlike ParticleNet, transformer-based models use attention mechanisms to capture long-range particle correlations and global jet structure.

GloParT uses large-scale pretraining on millions of jets to build general-purpose jet representations that can later be fine-tuned for specific analyses. Due to the complexity of the training setup and dataset requirements, this part was not fully completed within the scope of the present project, but it remains an important future direction for jet-tagging research. The transformer-based architecture explored in GloParT is shown in Figure 9.

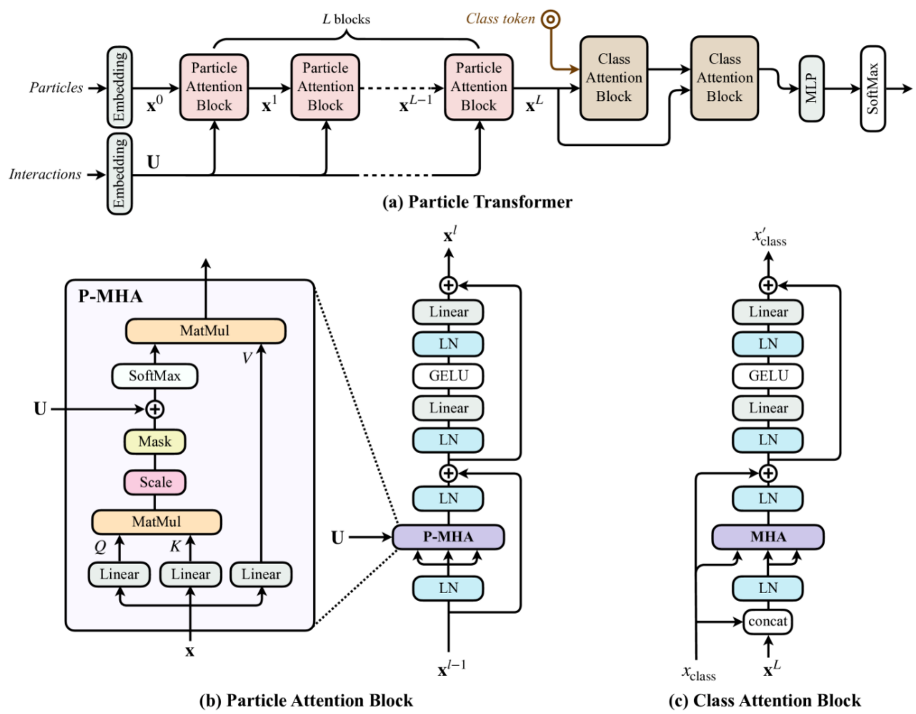


Figure 9: Architecture of Particle Transformer and related attention blocks used in GloParT. Transformer-based architectures aim to capture long-range particle correlations through attention mechanisms. Adapted from Qu et al. [5].

## 6 Conclusion

In this project, the ParticleNet jet-tagging framework was successfully reproduced and studied using CMS datasets and documentation. The graph-based particle cloud representation was shown to provide strong performance compared to traditional MLP and CNN approaches.

The project additionally provided practical experience with graph neural networks, dynamic graph construction, hyperparameter tuning, and TensorBoard-based training analysis. Future work may extend these studies toward transformer-based architectures such as GloParT and Particle Transformer models.

## References

[1] H. Qu and L. Gouskos, "Jet Tagging via Particle Clouds," Phys. Rev. D 101, 056019 (2020), arXiv:1902.08570.

[2] CMS Machine Learning Group, "ParticleNet Documentation," `https://cms-ml.github.io/documentation/inference/particlenet.html`

[3] Y. Wang et al., "Dynamic Graph CNN for Learning on Point Clouds," ACM TOG 38 (2019), arXiv:1801.07829.

[4] M. Zaheer et al., "Deep Sets," NeurIPS 2017, arXiv:1703.06114.

[5] H. Qu et al., "Particle Transformer for Jet Tagging," JHEP 2022, arXiv:2202.03772.

[6] CMS Collaboration, "Identification of heavy-flavour jets with the CMS detector in pp collisions at 13 TeV," JINST 13 (2018) P05011, arXiv:1707.08966.